Типичные ошибки

Типичные ошибки

Зачем об этом говорить?

Ошибки в архитектуре и коде - это нормально. Особенно для первого проекта. Проблема не в том, что они случаются, а в том, что некоторые из них незаметны на ранних этапах. Код работает, игра запускается. Но через пару недель оказывается, что добавить новую фичу невозможно, не сломав всё остальное. Или что баг в одной системе ломает три другие.

Этот раздел - коллекция граблей, на которые наступают чаще всего. Некоторые из них уже упоминались в других разделах, здесь они собраны в одном месте.

God Object (Божественный объект)

Как выглядит

Один класс, который делает всё. Обрабатывает ввод, двигает персонажа, считает физику, рисует графику, проигрывает звуки, сохраняет игру. Обычно называется `Game`, `GameManager` или `MainController` и занимает сотни (или тысячи) строк.

Python | C#

```python
class Game:
    def __init__(self):
        self.player_x = 100
        self.player_y = 200
        self.player_health = 100
        self.enemies = []
        self.inventory = []
        self.score = 0
        self.is_paused = False
        self.sound_volume = 0.5
        self.save_path = "save.json"
        # ... 30 more fields

    def update(self):
        self.handle_input()
        self.move_player()
        self.move_enemies()
        self.check_collisions()
```

```python
        self.update_physics()
        self.update_ui()
        self.check_save()
        self.play_sounds()
        # ... 10 more methods

    def handle_input(self):
        keys = pygame.key.get_pressed()
        if keys[pygame.K_w]:
            self.player_y -= 5
        if keys[pygame.K_e]:
            self.open_inventory()
        if keys[pygame.K_F5]:
            self.save_game()
        # 20 more if-statements...
```

Почему это плохо

- **Навигация**: где искать код столкновений? Где-то среди 1500 строк. Удачи.
- **Изменения**: хотите изменить физику? Метод `update_physics` использует 10 полей из того же класса. Поменяли одно - сломалось другое.
- **Тестирование**: невозможно проверить работу одной системы отдельно от всех остальных.

Как исправить

Грубо говоря - разбить на отдельные системы. Для этого мы и разбирали архитектуру, MVC, ECS. У каждой системы - своя зона ответственности. Game-класс остаётся, но он только **координирует**: создаёт системы, вызывает update у каждой.

Python C#

```python
class Game:
    def __init__(self):
        self.input = InputManager()
        self.physics = PhysicsSystem()
        self.renderer = Renderer()
        self.audio = AudioSystem()

    def update(self, delta_time):
        self.input.update()
        self.physics.update(delta_time)

    def draw(self):
        self.renderer.draw()
```

Т.е. Game знает **о** системах, но не знает **что они делают внутри**.

Логика в Draw

Как выглядит

Код, который меняет состояние игры, находится в методе отрисовки.

Python C#

```python
def draw(self, screen):
    # rendering... wait, why is game logic here?
    self.player.x += self.player.speed
    if self.player.x > 800:
        self.player.x = 0

    pygame.draw.rect(screen, BLUE,
        (self.player.x, self.player.y, 32, 32))
```

Почему это плохо

- `Draw` может вызываться **не каждый кадр** или **чаще, чем Update**. Логика начнёт зависеть от частоты отрисовки.
- Нарушается разделение ответственности. Draw должен **отображать** текущее состояние, а не **менять** его.
- Баги от этого крайне неочевидные - "персонаж двигается быстрее на мощном компьютере" или "физика ломается при включении вертикальной синхронизации".

Как исправить

Ок, правило простое: всё, что меняет состояние игры, - в `Update`. Всё, что рисует, - в `Draw`. Без исключений.

Флаговый ад

Как выглядит

Десятки булевых переменных, определяющих состояние объекта. Мы уже видели это в разделе про конечные автоматы:

Python C#

```python
is_jumping = False
is_drinking = False
```

```python
is_attacking = False
is_blocking = False
is_stunned = False
is_dead = False

if is_jumping and not is_drinking and not is_dead and not is_stunned:
    if is_attacking and not is_blocking:
        # ...
```

Почему это плохо

- Количество комбинаций растёт экспоненциально. 6 флагов = 64 возможных состояний. Вы точно хотите проверять все?
- Невалидные комбинации: `is_dead = True` и `is_jumping = True` одновременно. Кто-то забыл сбросить флаг.
- Добавление нового действия = правка десятков условий по всему коду.

Как исправить

Конечные автоматы. Объект всегда находится в **одном** состоянии. Переходы между состояниями определены явно.

Магические числа

Как выглядит

Числа в коде без объяснения, что они означают.

Python | C#

```python
if player.x > 1920:
    player.x = 0
player.health -= 15
if distance < 200:
    enemy.attack()
```

Хм. Что такое 1920? Ширина экрана? А если экран другой? Что такое 15? Урон от чего? Почему 200, а не 300?

Почему это плохо

- Через неделю вы не вспомните, что значит число.
- Если одно и то же число используется в 10 местах, и вы хотите его изменить - придётся найти все 10. И не перепутать с другими числами, которые совпали случайно.

Как исправить

Вынести в константы или настройки.

Python C#

```python
SCREEN_WIDTH = 1920
ENEMY_BASE_DAMAGE = 15
ENEMY_AGGRO_RANGE = 200

if player.x > SCREEN_WIDTH:
    player.x = 0
player.health -= ENEMY_BASE_DAMAGE
if distance < ENEMY_AGGRO_RANGE:
    enemy.attack()
```

Теперь код читается как текст, а изменение значения - в одном месте.

Всё в одном файле

Как выглядит

Весь код игры - в одном файле. 2000+ строк. Player, Enemy, InputManager, UI, Physics, SaveSystem - всё рядом.

Почему это плохо

- Навигация по файлу - мучение.
- Два человека не могут параллельно работать над разными системами (merge-конфликты).
- Сложно понять структуру проекта, глядя на один файл.

Как исправить

Один класс (или один логический модуль) - один файл. Группировка по папкам:

Python C#

```
src/
  entities/
    player.py
    enemy.py
  systems/
    physics.py
    input_manager.py
    audio.py
```

```
ui/
  health_bar.py
  menu.py
game.py
```

Ну, в начале кажется, что файлов слишком много. Но когда проект вырастет - вы скажете себе спасибо.

Копипаст вместо обобщения

Как выглядит

Один и тот же код повторяется в нескольких местах с минимальными изменениями.

Python C#

```python
def update_enemy_goblin(goblin):
    goblin.x += goblin.speed
    if goblin.health <= 0:
        goblin.alive = False

def update_enemy_skeleton(skeleton):
    skeleton.x += skeleton.speed
    if skeleton.health <= 0:
        skeleton.alive = False

def update_enemy_dragon(dragon):
    dragon.x += dragon.speed
    if dragon.health <= 0:
        dragon.alive = False
```

Почему это плохо

Нашли баг в логике перемещения? Исправляйте в трёх (или тридцати) местах. Забыли одно - получили непонятное поведение.

Как исправить

Обобщить. Если враги ведут себя одинаково - используйте один метод (или базовый класс, или компонент).

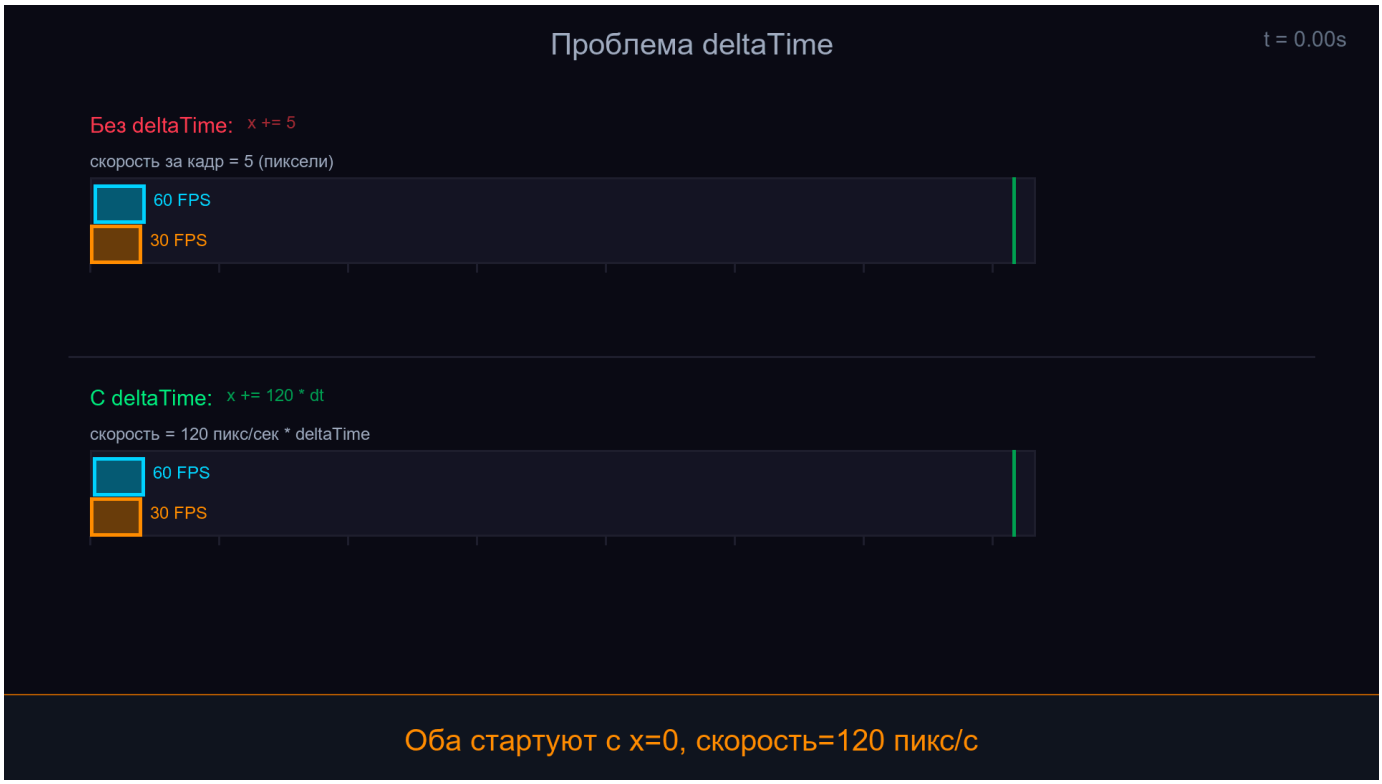
Python C#

```python
def update_enemy(enemy):
    enemy.x += enemy.speed
    if enemy.health <= 0:
        enemy.alive = False

for enemy in enemies:
    update_enemy(enemy)
```

Но не стоит обобщать слишком рано. Если у вас два похожих куска кода в разных классах и вы подозреваете, что они будут расходиться - дублирование может быть оправданным. Обобщайте, когда паттерн повторяется три и более раз.

Игнорирование deltaTime



Как выглядит

Перемещение объектов на фиксированное расстояние за кадр, без учёта времени.

Python C#

```python
# in update:

```

```
    player.x += 5  # 5 pixels per frame
```

Почему это плохо

Допустим, на компьютере с 60 FPS игрок проходит 300 пикселей в секунду. На компьютере с 120 FPS - 600 пикселей в секунду. На слабом ноутбуке с 30 FPS - 150. Скорость игры зависит от производительности компьютера.

Как исправить

Использовать deltaTime - время, прошедшее с прошлого кадра. Мы подробно разбирали это в разделе про игровой цикл.

Python | C#

```
speed = 300  # pixels per second
player.x += speed * delta_time
```

Теперь скорость одинакова на любом компьютере: 300 пикселей в секунду, независимо от FPS.

Итого

Ошибка	Суть	Решение
God Object	Один класс делает всё	Разбить на системы (архитектура)
Логика в Draw	Изменение состояния при отрисовке	Логика в Update, отрисовка в Draw
Флаговый ад	Десятки bool-ов и if-else	Конечные автоматы
Магические числа	Непонятные числа в коде	Константы и настройки
Один файл	Весь код в одном месте	Файл на класс, папки по модулям
Копипаст	Дублирование кода	Обобщение, базовые классы, компоненты
Без deltaTime	Скорость зависит от FPS	Умножение на deltaTime

Что дальше?

Что	Где подробнее
Инструменты для поиска ошибок	Инструменты отладки
Порядок обновления и физический цикл	Порядок обновления

Powered by Wiki.js